

CSCI 4407 IA 1 - Eduardo Galvez

Assignment Tasks

Task 1: Understanding the Operations

Addition modulo 2^{32}

The underlying purpose of the Addition modulo is that it allows us to work within the constraint of a 32 range which is the size of a word unit, this allows it to run fast on any processor since processors are primarily either 32-bit or 64-bit thus beneficial in cryptography from a performance standpoint. This operation is denoted as $a+b \bmod(2^{32})$ which takes the sum of two integers and by taking the mod 2^{32} creates a value that is restricted to the 32-bit range. Another benefit is that it allows us to mix two values into a non-linear mixing of bits.

XOR operation

Exclusive OR is only true when the two inputs differ or false when the two inputs are the same. The purpose of this in cryptography is that it allows us a way to combine two values into a unique value. This is of importance since it is an operation that is reversible by taking the inverse so we are able to recover the original value. What makes this valuable is that it is a nondestructive efficient manner to mix bits.

Bit rotation (left rotation)

The standard bitwise left shift in programming can be used to double the value of the input by the amount of shifts you would like, 3 shifts would be $n \cdot 2^3$ and so forth. With bit rotation of left rotation we are preserving the data but rotating the leftmost bit to the right. The new value will be completely independent of the original value but with proper shift tracking we are able to mask the original value to something different. This is also of value in cryptography as it gives us a way to obscure information while being able to reverse the operation to retrieve the original text.

All of these operations independently allow us to obfuscate data in a fairly one dimensional format but these methods are commonly known and would be easily brute forced by any cryptographic novice. What can truly be something of note is what happens if you layer these operations together. When you combine these operations it is called an ARX operation and makes it very difficult to analyse. It creates a strong diffusion that makes it incredibly resistant to standard cracking methods while being very efficient. This is the genius that is behind ChaCha20.

Task 2 - Eduardo

$a = 0x11111111$ $b = 0x01020304$ $c = 0x9b8d6f43$ $d = 0x01234567$

mod add xor left rotate

$a += b$; $d^1 = a$; $d \lll 16$ OPERATION #1

$0x11111111$
 $+ 0x01020304$
 $a = 0x12131415$

$0x01234567$ d
 $\oplus 0x12131415$ a
 $0x13305172$ d
 $d = 0x13305172$

$0x13305172$ d
 $\oplus 0x12131415$ a
 $0x51721330$ d
 $d = 0x51721330$

$c \oplus d$; $b^1 = c$; $b \lll 12$ OPERATION #2

$0x9b8d6f43$
 $+ 0x51721330$
 $c = 0xecff8273$

$0x01020304$
 $\oplus 0xecff8273$
 $b = 0xedfd8177$

$b = 0xedfd8177$
 $\lll 12$
 $b = 0xd8177edf$

$a += b$; $d^1 = a$; $d \lll 8$ OPERATION #3

$0x12131415$
 $\oplus 0x51721330$
 $a = 0xea2a92f4$

$0x51721330$ d
 $\oplus 0xea2a92f4$ a
 $0xb5881c4b$ b

$0xb5881c4b$ b
 $\lll 8$
 $b = 0x5881c4bb$

$c += d$; $b^1 = c$; $b \lll 7$

$0xecff8273$
 $+ 0x5881c4bb$
 $c = 0x4581472e$

$0xd8177edf$ b
 $\oplus 0x4581472e$ c
 $0x9d9639f1$ b

$0x9d9639f1$ b
 $\lll 7$
 $b = 0xc81cf8ce$

$a = 0xea2a92f4$, $b = 0xc81cf8ce$, $c = 0x4581472e$
 $d = 0x5881c4bb$

a = 0xea2a92f4, b = 0xcb1cf8ce, c = 0x4581472e, d = 0x5881c4bb

Note: a XOR hexadecimal table was used to easily compute the XOR operations

Task 3: Implementation of the Quarter-Round

```
def rot32(x, n):
```

```
    """
```

```
    Left-rotate a 32-bit integer x by n bit positions
```

```
    """
```

```
    return ((x << n) | (x >> (32 - n))) & 0xFFFFFFFF
```

```
def quarter_round(a, b, c, d):
```

```
    """
```

```
    Apply the ChaCha20 quarter-round transformation to four 32-bit words
```

```
    All additions are modulo  $2^{32}$ 
```

```
    If result exceeds 32 bits only the least significant 32 bits are kept
```

```
    """
```

```
    # Operation 1
```

```
    a = (a + b) & 0xFFFFFFFF # modular addition
```

```
    d = d ^ a                # XOR
```

```
    d = rot32(d, 16)         # left rotation by 16
```

```
    # Operation 2
```

```
    c = (c + d) & 0xFFFFFFFF
```

```
    b = b ^ c
```

```
    b = rot32(b, 12)
```

```
    # Operation block 3
```

```
    a = (a + b) & 0xFFFFFFFF
```

```
    d = d ^ a
```

```
    d = rot32(d, 8)
```

```
    # Operation block 4
```

```
    c = (c + d) & 0xFFFFFFFF
```

```
    b = b ^ c
```

```
    b = rot32(b, 7)
```

```
    return a, b, c, d
```

```
a, b, c, d = quarter_round(
```

```
    0x11111111, # a
```

```
    0x01020304, # b
```

```
    0x9b8d6f43, # c
```

0x01234567) # d

```
print(f'a = {hex(a)}')
print(f'b = {hex(b)}')
print(f'c = {hex(c)}')
print(f'd = {hex(d)}')
```

```
1  a, b, c, d = quarter_round(
2      0x11111111, # a
3      0x01020304, # b
4      0x9b8d6f43, # c
5      0x01234567) # d
6
7  print(f'a = {hex(a)}')
8  print(f'b = {hex(b)}')
9  print(f'c = {hex(c)}')
10 print(f'd = {hex(d)}')
```

✓ 0.0s Python

a = 0xea2a92f4
b = 0xcb1cf8ce
c = 0x4581472e
d = 0x5881c4bb

The results do match my final manually computed results

Task 4: Diffusion Experiment

```
1  a, b, c, d = quarter_round(
2      0x11111101, # a
3      0x01020304, # b
4      0x9b8d6f43, # c
5      0x01234567) # d
6
7  print(f'a = {hex(a)}')
8  print(f'b = {hex(b)}')
9  print(f'c = {hex(c)}')
10 print(f'd = {hex(d)}')
```

✓ 0.0s Python

a = 0xea2a92e3
b = 0xb3147876
c = 0x3571562e
d = 0x4881d3bb

Flipped one bit in the a input 0x11111111 to 0x11111101 and the results have a words that seem to match but the overall output values do differ from the original inputs from a value perspective at least.

a = 0xea2a92f4
a' = 0xea2a92e3

b = 0xcb1cf8ce
b' = 0xb3147876

c = 0x4581472e
c' = 0x3571562e

d = 0x5881c4bb
d' = 0x4881d3bb

```
ethos@DESKTOP-J4VSN7B:/mnt/c/Users/Ethos/source/repos/CSCI4407$ python3 IA1/IA1Task4.py
a = 111010100010101001001011110100
a' = 111010100010101001001011100011

b = 11001011000111001111100011001110
b' = 10110011000101000111100001110110

c = 010001011000000010100011100101110
c' = 001101010111000101011000101110

d = 010110001000000011100010010111011
d' = 010010001000000011101001110111011
```

Number of bits changed:

- a: 4 bits changed
- b: 10 bits changed
- c: 9 bits changed
- d: 5 bits changed

Looking at the actual bits instead of the hexadecimal values the two are actually a lot more similar than I would have expected. While ChaCha does a good job at completely changing the output from the input it may be susceptible to

Task 5: Critical Reflection

Why does ChaCha use multiple rotations (16, 12, 8, and 7)?

With this combination of rotations it ensures that all the characters get shuffled. The biggest advantage with this selection of numbers is that it ensures that everything is getting shifted, if

you were for example to only have 16 shift rotations it would cause it to keep swapping the first half and the second half of the bitstream which might be OK with all the other operations but the rotations being different amounts is a clever way to easily create more diffusion.

How does the quarter-round contribute to diffusion in the cipher?

Each quarter round has each of the four inputs update twice over, this is over the course of 4 additions, 4 XORs, and 4 rotations. Since these operations are done with the updated values the bits become more thoroughly mixed the deeper into the operations we get into, it is almost similar to an avalanche effect since a single bit change is able to have a significant change to the overall output FOR ONLY ONE BIT CHANGE.

Why are modular addition, XOR, and rotations effective together in ARX-based cryptographic designs?

Addition, rotations, and XOR are the basis for ARX since they are ways to modify an input in using operations that are independent from each other. While independently these operations are fairly weak since they can be easily cracked but when layered on top of each other they are able to cover the potential shortcomings or patterns developed from one alone, more so since each operation is done 4 times with the updated values of each round.

What is (a, b, c, and d) and what are their influence?

In the full ChaCha20 cipher, the 4x4 state matrix contains 16 32-bit words. When quarter round is used it selects 4 of these 16 words. The labels them a, b, c, d as local names for whichever four words are being mixed. They each affect one another: a and c are updated via addition with b and d respectively, while b and d are updated via XOR with c and a. After one full quarter round, a change in any single input word will have influenced all four output words through this chain of additions and XORs.

Research Exploration Tasks

Task 6: Avalanche Effect Analysis

Task 6: Avalanche Effect Analysis

Flip each of the 32 bit positions of input 'a' one at a time

Original inputs: a=0x11111111 b=0x1020304 c=0x9b8d6f43 d=0x1234567
Original outputs: a=0xea2a92f4 b=0xcb1cf8ce c=0x4581472e d=0x5881c4bb

Bit flipped	a bits	b bits	c bits	d bits	Total
bit 0	4	17	10	5	36
bit 1	4	16	15	5	40
bit 2	5	14	8	7	34
bit 3	4	14	8	5	31
bit 4	4	10	9	5	28
bit 5	6	12	11	7	36
bit 6	4	11	10	5	30
bit 7	4	12	10	5	31
bit 8	3	13	9	6	31
bit 9	6	7	6	7	26
bit 10	3	8	6	5	22
bit 11	3	7	5	4	19
bit 12	3	10	8	4	25
bit 13	4	9	6	5	24
bit 14	4	7	6	5	22
bit 15	4	9	8	5	26
bit 16	5	14	11	6	36
bit 17	4	11	9	6	30
bit 18	3	9	8	4	24
bit 19	4	10	9	5	28
bit 20	4	8	7	5	24
bit 21	3	6	5	4	18
bit 22	2	7	5	3	17
bit 23	3	8	7	4	22
bit 24	5	15	8	7	35
bit 25	4	13	11	6	34
bit 26	2	8	7	3	20
bit 27	3	7	6	4	20
bit 28	4	14	10	5	33
bit 29	5	11	10	6	32
bit 30	3	10	9	4	26
bit 31	5	17	11	6	39

Summary across all 32 single-bit flips of input 'a':

Average bits changed : 28.09 / 128

Minimum bits changed : 17

Maximum bits changed : 40

This task while similar to Task 4 forces us to prove our analysis. In flipping through the bits of a one bit at a time we are able to show that with a single bit flip it causes at an average a 28% change in the overall 32 bits. For a single bit flip this is a significant amount of diffusion that

showcases the strengths of ChaCha's design. So this is a large amount of diffusion for a single bit shift it does not change a majority of the bits which MAY be a weak point since it does have a significant amount of overlap between the two to potentially identify a pattern.

Task 7: Rotation Constant Exploration

```
=====
Task 6: Avalanche Effect Analysis (Modified for Task 7 with 15 rotation)
Flip each of the 32 bit positions of input 'a' one at a time
=====
```

```
Original inputs: a=0x11111111 b=0x1020304 c=0x9b8d6f43 d=0x1234567
Original outputs: a=0x59d11069 b=0xeecb0c35 c=0x2c606a4c d=0x6819f171
```

Bit flipped	a bits	b bits	c bits	d bits	Total
bit 0	4	8	7	5	24
bit 1	4	13	11	5	33
bit 2	3	12	10	5	30
bit 3	4	9	7	5	25
bit 4	4	13	9	5	31
bit 5	4	14	11	5	34
bit 6	5	13	11	6	35
bit 7	2	14	12	3	31
bit 8	10	13	11	13	47
bit 9	3	8	7	4	22
bit 10	3	15	12	5	35
bit 11	3	15	13	4	35
bit 12	9	13	9	10	41
bit 13	2	10	9	3	24
bit 14	5	11	9	6	31
bit 15	3	8	6	4	21
bit 16	3	6	5	4	18
bit 17	5	11	9	7	32
bit 18	4	11	9	5	29
bit 19	4	8	7	5	24
bit 20	3	7	6	4	20
bit 21	2	8	7	3	20
bit 22	5	6	5	6	22
bit 23	6	12	9	7	34
bit 24	3	12	13	5	33
bit 25	2	5	4	4	15
bit 26	3	10	9	4	26
bit 27	7	8	7	8	30
bit 28	2	9	8	3	22
bit 29	3	10	6	4	23
bit 30	3	12	9	4	28
bit 31	2	10	8	3	23

```
-----
Summary across all 32 single-bit flips of input 'a':
Average bits changed : 28.06 / 128
Minimum bits changed : 15
Maximum bits changed : 47
```

While the bits did change the average bit change actually only changed marginally 28.09/128 bits to 28.06/128 bits. So we do observe a 0.03 drop in bit change but maintain an overall average of 21.9% bit change. From the ChaCha paper they did experiment with several rotation sets (7, 9, 13, 18), (16, 12, 8, 7), and (7, 9, 13, 18) but (16, 12, 8, 7) turned out to be marginally better than all the others, the margins were tiny but might as well just use the best of the bunch, any optimization gains is good.

Task 8: Input Word Influence Analysis

Task 8: Input Word Influence Analysis

Flip bit 0 of each input word in turn, hold all others constant

Original inputs: a=0x11111111 b=0x1020304 c=0x9b8d6f43 d=0x1234567
Original outputs: a=0xea2a92f4 b=0xcb1cf8ce c=0x4581472e d=0x5881c4bb

Word flipped	out-a bits	out-b bits	out-c bits	out-d bits	Total	All 4 affected?
bit 0 of a	4	17	10	5	36	Yes

Flipping bit 0 of a:

a = 111010100010101010010011110100
a' = 001110100010101010010011110100 (4 bits changed)
^ ^ ^ ^
b = 11001011000111001111100011001110
b' = 010010100011000000111100110111 (17 bits changed)
^
c = 01000101100000010100011100101110
c' = 0100011010000001001100110111110 (10 bits changed)
^
d = 01011000100000011100010010111011
d' = 01011001100000011110010001101011 (5 bits changed)
^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^

bit 0 of b	4	10	8	6	28	Yes
------------	---	----	---	---	----	-----

Flipping bit 0 of b:

a = 111010100010101010010011110100
a' = 11011010001010101000001011110101 (4 bits changed)
^
b = 11001011000111001111100011001110
b' = 01000011100101001001000011000000 (10 bits changed)
^
c = 01000101100000010100011100101110
c' = 01001000100100000100011111111110 (8 bits changed)
^
d = 01011000100000011100010010111011
d' = 01011011100100011100010110001011 (6 bits changed)
^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^

bit 0 of c	1	4	3	1	9	Yes
------------	---	---	---	---	---	-----

Flipping bit 0 of c:

a = 111010100010101010010011110100
a' = 11101010001010101000001011110100 (1 bits changed)
^
b = 11001011000111001111100011001110
b' = 11000011000101001111100101001110 (4 bits changed)
^
c = 01000101100000010100011100101110
c' = 010001011001000101000111100101101 (3 bits changed)
^
d = 01011000100000011100010010111011
d' = 01011000100100011100010010111011 (1 bits changed)
^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^

bit 0 of d	5	16	9	6	36	Yes
------------	---	----	---	---	----	-----

Flipping bit 0 of d:

a = 111010100010101010010011110100
a' = 001110100010101010010011010101 (5 bits changed)
^
b = 11001011000111001111100011001110
b' = 0100101000110010001111100110111 (16 bits changed)
^
c = 01000101100000010100011100101110
c' = 01000110100000010011001111101110 (9 bits changed)
^
d = 01011000100000011100010010111011
d' = 01011001100000011110010101101011 (6 bits changed)
^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^

Typically it does seem to reflect what we saw in changing bits in a lone. The biggest note of difference is that flipping the bit in c caused the least amount of diffusion as only 9 bits total changed. This may be attributed to the fact that c is the only part that does not have any part in the first operation thus the first operation would not change when c is adjusted, another weakness of this implementation.

Task 9: Reduced Transformation Analysis

```
=====
Task 9: Reduced Transformation Analysis
Observe how diffusion grows after each operation block
Original a=0x11111111 Modified a=0x11111110 (bit 0 flipped)
=====
```

```
After Block 1 a+=b; d^=a; d<<<16
State (original): a=0x12131415 b=0x1020304 c=0x9b8d6f43 d=0x51721330
State (modified): a=0x12131414 b=0x1020304 c=0x9b8d6f43 d=0x51731330
Bits changed - a:1 b:0 c:0 d:1 | total: 2/128 | words affected: 2/4
a = 00010010000100110001010000010101
a' = 00010010000100110001010000010100
      ^
b = 00000001000000100000001100000100
b' = 00000001000000100000001100000100

c = 10011011100011010110111101000011
c' = 10011011100011010110111101000011

d = 01010001011100100001001100110000
d' = 01010001011100110001001100110000
      ^
```

```
After Block 2 c+=d; b^=c; b<<<12
State (original): a=0x12131415 b=0xd8177edf c=0xecff8273 d=0x51721330
State (modified): a=0x12131414 b=0x28177ec0 c=0xed008273 d=0x51731330
Bits changed - a:1 b:9 c:9 d:1 | total: 20/128 | words affected: 4/4
a = 00010010000100110001010000010101
a' = 00010010000100110001010000010100
      ^
b = 11011000000101110111111011011111
b' = 00101000000101110111111011000000
      ^^^^^ ^^^^^
c = 11101100111111111000001001110011
c' = 11101101000000001000001001110011
      ^^^^^^^^^
d = 01010001011100100001001100110000
d' = 01010001011100110001001100110000
      ^
```

```
After Block 3 a+=b; d^=a; d<<<8
State (original): a=0xea2a92f4 b=0xd8177edf c=0xecff8273 d=0x5881c4bb
State (modified): a=0x3a2a92d4 b=0x28177ec0 c=0xed008273 d=0x5981e46b
Bits changed - a:4 b:9 c:9 d:5 | total: 27/128 | words affected: 4/4
a = 11101010001010101001001011110100
a' = 00111010001010101001001011010100
      ^^ ^ ^
b = 11011000000101110111111011011111
b' = 00101000000101110111111011000000
      ^^^^^ ^^^^^
c = 11101100111111111000001001110011
c' = 11101101000000001000001001110011
      ^^^^^^^^^
d = 01011000100000011100010010111011
d' = 01011001100000011110010001101011
      ^ ^ ^ ^
```

```
After Block 4 c+=d; b^=c; b<<<7
State (original): a=0xea2a92f4 b=0xcb1cf8ce c=0x4581472e d=0x5881c4bb
State (modified): a=0x3a2a92d4 b=0x4a8c0f37 c=0x468266de d=0x5981e46b
Bits changed - a:4 b:17 c:10 d:5 | total: 36/128 | words affected: 4/4
a = 11101010001010101001001011110100
a' = 00111010001010101001001011010100
      ^^ ^ ^
b = 11001011000111001111100011001110
b' = 01001010100011000000111100110111
      ^ ^^ ^ ^^^^^ ^^^^^
c = 01000101100000010100011100101110
c' = 01000110100000100110011011011110
      ^^ ^^ ^ ^^^^^
d = 01011000100000011100010010111011
d' = 01011001100000011110010001101011
      ^ ^ ^ ^
```


From what we can see diffusion is actually very little (2 bits change) after the first operation block. When we get through the second operation we do start to see significant amount of diffusion (20 bits change). At operation 3 and 4 we do see more bits change but this second operation would be the largest jump, it would make sense since we are actually getting all the bits changed after the second operation since c and b do not change until then.

Task 10: Reflection on ARX Cryptographic Design

The three operations that make up the ARX design Addition modulo 2^{32} , XOR, and bit rotation are each valuable in isolation, but what makes them truly powerful is what happens when you layer them together.

Addition modulo 2^{32} works within the constraint of a 32-bit range, which is the size of a word unit. This allows it to run fast on any processor since processors are primarily either 32-bit or 64-bit, making it beneficial in cryptography from a performance standpoint. Beyond performance, it introduces a non-linear mixing of bits through carry propagation when a carry ripples upward through the addition, higher bits are affected in a way that is difficult to predict or reverse analytically.

XOR is only true when the two inputs differ and false when they are the same. What makes this valuable in cryptographic design is that it is a nondestructive and efficient way to mix bits it is fully reversible by applying the same operation again, meaning no information is lost. This reversibility is important because it means the cipher can be decrypted, while the bit-mixing behaviour ensures that the relationship between input and output is thoroughly obscured.

Bit rotation preserves all of the data in a word but repositions it, moving bits that were on one end of the word to the other. Unlike a standard left shift which discards bits, rotation loses nothing. This is valuable in cryptography because it gives us a way to scatter the influence of each bit across different positions, making it harder to trace where any particular bit came from, while still being fully reversible.

Independently, each of these operations obfuscates data in a fairly one dimensional format and the methods are well understood. However, when you combine them into an ARX construction, something more significant emerges. The addition and XOR operations work against each other's algebraic structure addition carries bits upward in ways that XOR cannot predict, and XOR mixes bits in ways that addition cannot undo cleanly. Rotation then scatters those mixed bits into new positions before the next addition picks them up, amplifying the confusion further with every step. The result is a construction that is incredibly resistant to standard cryptanalytic methods while remaining computationally efficient on virtually any hardware. This interaction between simple operations producing complex, hard to analyse behaviour is the core insight

behind ChaCha20, and it is what allows a cipher built from nothing more than integer addition, XOR, and bit rotation to be trusted in securing real-world communications.